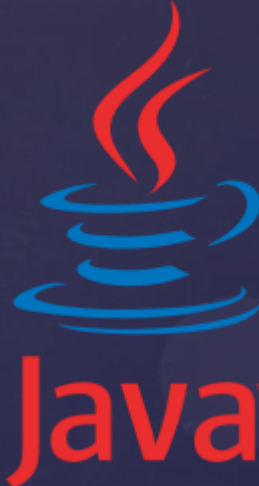




Programmation Orientée Objet



Katy SAINTIN
Octobre 2016



Pourquoi autant de succès ?

Les objets et les classes

Les pointeurs et les références

Les classes

- ✓ *Introduction*
- ✓ *Les champs ou données de la classe*
- ✓ *Les méthodes de classe*
- ✓ ***L'encapsulation***
- ✓ *Le constructeur*
- ✓ *Mot clé « this »*
- ✓ *Les accesseurs*
- ✓ *La méthode toString()*

Pourquoi autant de succès ?

Parce qu'elle nous aide à mieux organiser la complexité inhérente aux logiciels et celle caractéristique du monde réel à modéliser.

Grâce à la POO les logiciels sont :

- Plus maintenables
- Modulaires
- Souples et évolutifs
- Optimisés



procédural



objet

Avant, les développeurs concentraient leurs efforts sur les algorithmes. Avec les objets, le travail sur l'algorithme est minimisé. Leur souplesse permet aux développeurs d'aller beaucoup plus loin dans la réalisation et la conception de programmes "intelligents" et modulaires.

Un objet, c'est une entité réutilisable, qui se caractérise par :

- Un **état** : dans lequel l'objet se trouve.
- Un **comportement** : ce que l'objet est capable de faire.
- Une **identité unique** : ce qui distingue l'objet d'un autre.

Une classe est un **modèle** ou une **maquette** à partir de laquelle les objets vont être construits.

Ne jamais confondre une classe et un objet. L'objet est une **instance** de la classe qui représente son modèle.

Classe

Recette Pizza



Instance 1



Etat : Bien cuite

Identité : Pizza n°1

Comportement : Cuire

Objet

Instance 2



Etat : Crue

Identité : Pizza n°2

Comportement : Cuire

Instance 3



Etat : Cuite

Identité : Pizza n°3

Comportement : Cuire

Rappel : Objet, classe, pointeur et référence

Objet & classe

- ✓ *Objet* : Ma pizza préférée « Bella Italia »
- ✓ *Classe* : Recette de ma pizza

L'objet pizza contient beaucoup d'ingrédient et est donc volumineuse et difficile à recopier.



Pointeur & référence

- ✓ *Pointeur* : Adresse de la pizzeria qui fait ma pizza préférée, facile à recopier.(C++)
- ✓ *Référence* : Nom de la pizzeria « Chez Tony » facile à utiliser.(Java)

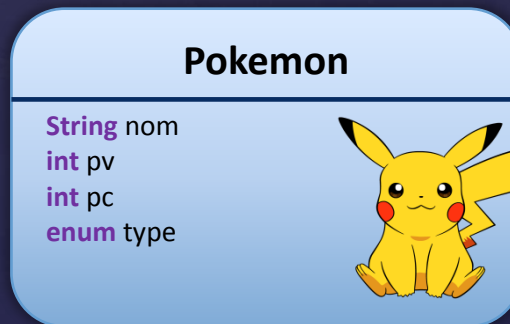


Les champs ou données d'une classe

Une classe peut contenir des données qui peuvent être : des objets ou des types primitifs. Les valeurs affectées à ces champs représente l'état d'un objet.

Prenons, l'exemple d'un objet Pokémon, ces attributs sont :

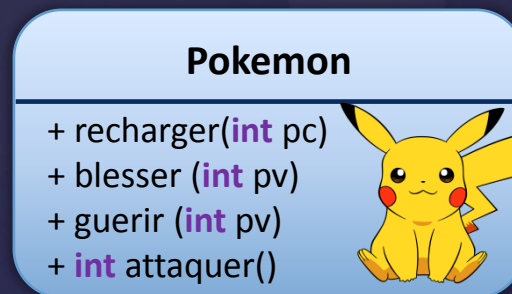
- ✓ **String** nom : un nom
- ✓ **int** pc : ses points de combat
- ✓ **int** pv : ses points de vie
- ✓ **enum** type : un type parmi Normal, Electrik, Eau, Feu ...



Ces méthodes sont des fonctions appelables par un client et elles définissent le comportement ou une action de l'objet.

Toujours dans l'exemple de l'objet Pokémon, ce dernier peut :

- ✓ Se recharger : gagner des points de combat,
- ✓ Se blesser : perdre des points de vie
- ✓ Se guérir : gagner des points de vie
- ✓ Attaquer : Infliger des blessures à un autre Pokémon



Les méthodes peuvent accepter des données en paramètres mais peuvent également renvoyer des données.

L'encapsulation permet de définir des niveaux de visibilité des éléments de la classe. Cet accès aux données se définit par le biais de « modifieurs » parmi les choix suivants :

public : donnée accessible à toutes les classes.

protected : donnée réservée aux classes héritières

private : donnée limitée à la classe elle-même

sans spécification du « modifier » la classe est package **friendly**

Toujours dans l'exemple de l'objet Pokémon, on peut imaginer que :

- ✓ Les champs seront privées et accessibles via des accesseurs
- ✓ Les méthodes seront publiques car accessibles à l'utilisateur

1. Créer une classe Pokemon
2. Créer l'énumération Type {Normal, Electrik, Eau, Feu}
3. Créer les champs :
 - ✓ **int** pv (qui ne peut jamais être négatif)
 - ✓ **int** pc (qui ne peut jamais être négatif)
 - ✓ **String** nom
 - ✓ **Type** type (égale à Normal par défaut)
4. Créer les méthodes :
 - ✓ renommer (String nom) : qui change le nom
 - ✓ recharger(int pcPlus) : qui ajoute pcPlus à pc
 - ✓ guerir(int pvPlus) : qui ajoute pvPlus à pv
 - ✓ blesser(int pvMoins) : qui enlève pvMoins à pv
 - ✓ int Attaquer() : qui retourne 0 par défaut

TP n°1 : Classe Pokemon solution



The value of the field is not used

The assignment has no effect

```
1 package fr.ifa.pokemons;
2
3 public class Pokemon {
4
5     //Enumeration type de pokemon
6     public static enum Type {
7         Normal, Electrik, Eau, Feu
8     }
9
10    private String nom = "";
11    // Point de vie
12    private int pv = 0;
13    // Point de combat
14    private int pc = 0;
15    // Type de pokemon
16    private Type type = Type.Normal;
17
18    public void renommer(String nom){
19        nom = nom;
20    }
21
22    public void recharger(int pcPlus) {
23        pc = pc + pcPlus;
24    }
25
26    public void guerir(int pvPlus) {
27        pv = pv + pvPlus;
28    }
29
30    public void blesser(int pvMoins) {
31        pv = pv + pvMoins;
32    }
33
34    public int Attaquer() {
35        return 0;
36    }
37
38 }
```

Variables de classe

Variable d'instance

Variable locale

Le mot clé "this" sert à désigner l'objet courant de la classe dans laquelle elle est utilisée. On peut distinguer 3 cas d'utilisation :

1. Pour éviter la répétition des variables pour plus de lisibilité (cf TP précédent pour la méthode renommer. On pourrait nommer le paramètre autrement, mais à ce qu'il paraît, les informaticiens sont de gros flemmards. 🤪

```
public void renommer(String nom){  
    this.nom = nom;  
}
```

2. Pour envoyer la référence d'un objet en paramètre.

```
public void renommer(String nom){  
    this.nom = nom;  
    maMethode(this, nom);  
}
```

3. Pour appeler un constructeur de la classe.

```
public Pokemon() {  
    this("Pikachu");  
}  
  
public Pokemon(String nom) {  
    renommer(nom);  
}
```

Un constructeur est une méthode un peu particulière qui :

- ✓ porte le **même nom** que la classe
- ✓ **ne renvoie rien** (pas même void)
- ✓ ne pourra être exécuté **qu'une seule fois**.
- ✓ peut prendre des paramètres
- ✓ peut être surchargé
- ✓ qui se charge d'initialiser les valeurs des champs.

L'appel au constructeur se fait avec l'opérateur **new**, on appelle cela **l'instanciation**.

```
//Constructeur vide
public Pokemon(){
    this("", 0, 0, Type.Normal);
}

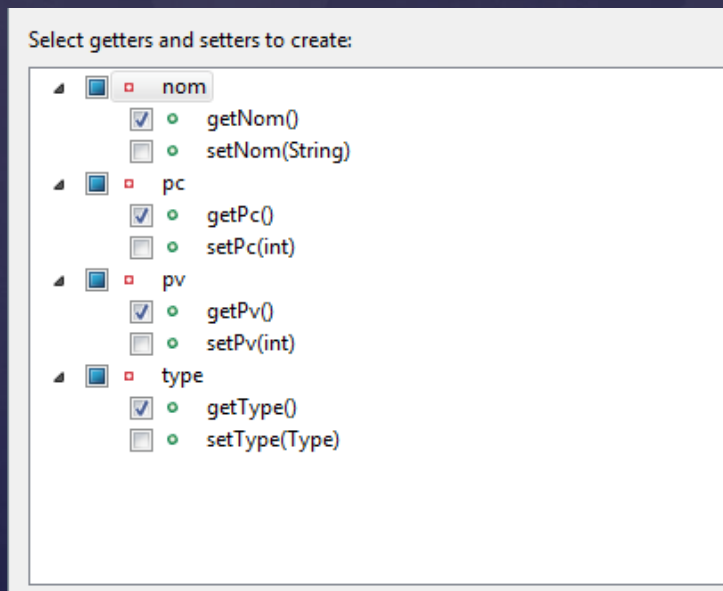
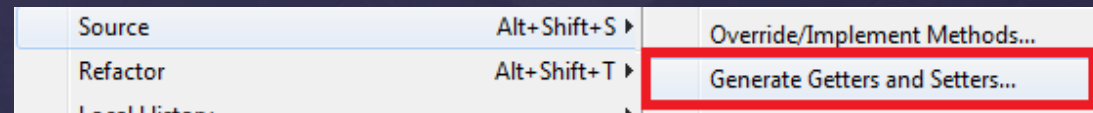
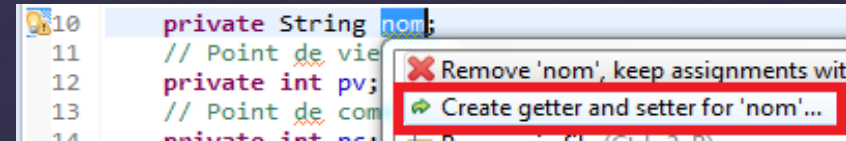
//Constructeur avec paramètre
public Pokemon(String nom, int pv, int pc, Type type){
    this.pv = 0;
    this.pc = 0;
    this.type = type;
    renommer(nom);
}

public static void main(String[] args) {
    //Creation d'un pokemon de base
    Pokemon pokemon = new Pokemon();
    //Creation d'un pokemon Pikachu
    Pokemon pikachu = new Pokemon("Pikachu", 45,423,Type.Electrik);
}
```



Les accesseurs sont des méthodes permettant l'accès et la modification des champs de classe (souvent privés), de manière **sécurisée**. Le standard java pose les règles de nommage suivant :

- Pour un champs non booléen maVariable
- La méthode d'accès (getter) sera **get**MaVariable()
- La méthode de modification (setter) sera **set**MaVariable()
- Pour un champs booléen monBooleen
- Le getter sera **is**MonBooleen()



```
public String getNom() {  
    return nom;  
}  
  
public int getPv() {  
    return pv;  
}  
  
public int getPc() {  
    return pc;  
}  
  
public Type getType() {  
    return type;  
}
```

La méthode toString()

La méthode `toString()` est une méthode de la classe `Objet` dont toutes classes *dérivent*. Elle est utilisée dans de nombreuses API java, en particulier la plus utile pour l'affichage d'un objet à savoir `System.out.println`.

- ✓ Sans *surcharge* la classe renvoie l'adresse de l'objet.
- ✓ Sinon le développeur peut spécifier le comportement souhaité (*polymorphisme*).

```
67 public static void main(String[] args) {
68     //Creation d'un pokemon de base
69     Pokemon pokemon = new Pokemon();
70     //Creation d'un pokemon Pikachu
71     Pokemon pikachu = new Pokemon("Pikachu", 45,423,Type.Electrik);
72
73     System.out.println("pokemon=" + pokemon);
74     System.out.println("pikachu=" + pikachu);
75 }
```

Console Problems

<terminated> Pokemon [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe (9 oct. 2016)

pokemon=fr.ifa.pokemons.Pokemon@2a139a55
pikachu=fr.ifa.pokemons.Pokemon@15db9742

```
67 @Override
68 public String toString() {
69     String toString = "Nom="+getNom() + "\n";
70     toString = toString + "Type="+getType() + "\n";
71     toString = toString + "Pc="+getPc() + "\n";
72     toString = toString + "Pv="+getPv() + "\n";
73     return toString;
74 }
75
76 public static void main(String[] args) {
77     //Creation d'un pokemon de base
78     Pokemon pokemon = new Pokemon();
79     //Creation d'un pokemon Pikachu
80     Pokemon pikachu = new Pokemon("Pikachu", 45,423,Type.Electrik);
81
82     System.out.println(pokemon);
83     System.out.println(pikachu);
84 }
```

Console Problems

<terminated> Pokemon [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe (9 oct. 2016)

Nom=
Type=Normal
Pc=0
Pv=0

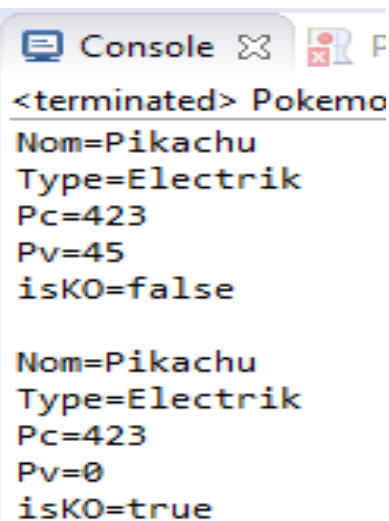
Nom=Pikachu
Type=Electrik
Pc=0
Pv=0

TP n°2 : Classe Pokemon avancée

1. Utiliser le mot clé **this** dans la méthode renommer
2. Rajouter un champs pvMax de type int
3. Créer des getter pour nom, pc, pv, type
4. Créer un constructeur avec nom, pc, pv et type en paramètre (utiliser le mot clé **this** pour affecter les champs)
5. Affecter pvMax = pv dans le constructeur
6. Faire en sorte que pv ne soit jamais négatif dans la méthode blesser.
7. Faire en sorte que pv ne soit jamais supérieur à pvMax dans la méthode guérir.
8. Rajouter une méthode isKO() qui retourne true si pv = 0
9. Surcharger la méthode toString afin d'afficher les valeurs de champs suivants : nom, pv, pc, type et isKO
10. Créer un main et instancier la classe Pokemon au choix si vous avez POKEMON GO, sinon reprendre l'exemple de Pikachu.

TP n°2 : Classe Pokemon avancée solution

```
1 package fr.ifa.pokemons;
2
3 public class Pokemon {
4
5     //Enumeration type de pokemon
6     public static enum Type {
7         Normal, Electrik, Eau, Feu
8     }
9
10    private String nom;
11    // Point de vie
12    private int pv;
13    // Point de combat
14    private int pc;
15    // Type de pokemon
16    private Type type;
17    // pvMax
18    private int pvMax;
19
20    //Constructeur avec paramètre
21    public Pokemon(final String nom, final int pv, final int pc, final Type type){
22        this.pv = pv;
23        this.pc = pc;
24        this.type = type;
25        this.pvMax = pv;
26        renommer(nom);
27    }
28
29    public String getNom() {
30        return nom;
31    }
32
33    public int getPv() {
34        return pv;
35    }
36
37    public int getPc() {
38        return pc;
39    }
40
41    public Type getType() {
42        return type;
43    }
44
```



```
<terminated> Pokemo
Nom=Pikachu
Type=Electrik
Pc=423
Pv=45
isKO=false

Nom=Pikachu
Type=Electrik
Pc=423
Pv=0
isKO=true
```

```
45    public void renommer(String nom){
46        if(nom != null && !nom.isEmpty()){
47            this.nom = nom;
48        }
49    }
50
51    public void guerir(int pvPlus) {
52        pv = pv + pvPlus > pvMax ? pvMax : pv;
53    }
54
55    public void blesser(int pvMoins) {
56        pv = pv - pvMoins < 0 ? 0 : pv;
57    }
58
59    public boolean isKO(){
60        return (getPv() == 0);
61    }
62
63    @Override
64    public String toString() {
65        String toString = "Nom="+getNom() + "\n";
66        toString = toString + "Type="+getType() + "\n";
67        toString = toString + "Pc="+getPc() + "\n";
68        toString = toString + "Pv="+getPv() + "\n";
69        toString = toString + "isKO="+isKO() + "\n";
70        return toString;
71    }
72
73
74    public static void main(String[] args) {
75        //Creation d'un pokemon Pikachu
76        Pokemon pikachu = new Pokemon("Pikachu", 45,423,Type.Electrik);
77        System.out.println(pikachu);
78        //Blesser pikachu
79        pikachu.blesser(100);
80        System.out.println(pikachu);
81    }

```